

Lec 49 U-Net for Denoising

Lec 49 U-Net for Denoising

Lecture	49 / 63 + intro
Duration	36:13
Captions	English (auto-generated)
Words	4,585
Video	oqA3Mjp2Nn4
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. Welcome back to the lecture series on fundamentals of generative AI and large language models. In today's lecture, we will discuss about the unit architecture, how it is used for denoising in DDPM. So, the contents of today's lecture goes like role of unit in DDPM, time embeddings, encoder block, bottleneck block, decoder block, residual blocks, self-attention, and the complete DDPM unit architecture.

[00:34] So, we will start with what is the role of unit in denoising diffusion probabilistic models. So, till now we have learned that in the forward process of DDPM, we add noise, you know, calculated amount of noise in each and every step such in such a way that after certain number of steps, we go to a from a clean image, we go to a completely noisy image. And in the reverse process, what we do is we start from a completely noisy image, go through several steps till we reach the clean image. And the neural network learns the process of denoising in the reverse or backward process. Okay? Now, the architecture that is used for this denoising or prediction of noise, how much noise is present in each

[01:42] and every image at various time steps, for predicting that noise, we use the unit architecture. So, the discussion today is all about the unit architecture, how it

is modified to be used in uh DDPM, what are the blocks that are present in the unit, what are the advantages of each and every block, and so on and so forth, okay? So, the role of unit in DDPM, so we know in DDPM, we have a noisy image X_T at a time step T .

[02:19] This will go through the unit architecture, and what will be the output of the unit architecture? It is the predicted noise, which we call as epsilon. Theta means it is the learned You know, theta is the parameters of the neural network. And this predicted noise depends on X_T and T . X_T is the noisy image at time instant T , which is input to the unit, and that time instant T . So, these two are the functions on which our noise prediction will depend.

[02:56] So, the main task of the unit is it learns to predict the Gaussian noise epsilon present in the noisy image X_T conditioned on time step T . Now, why unit is used? There Why not normal CNNs are used? We know unit captures the global context, the overall image structure, the information from the overall image is structured with is captured very beautifully by unit architecture, and it preserves the final details via skip connections. So, we all know these skip connections in unit architecture, they play a very significant role because they preserve the finer details.

[03:39] Whatever details get lost when we are down sampling, those are those comes back in the up comes up sampling part or the decoder part through the via the skip connections. So, these are the main two reasons why U-Net is preferred over other CNN architectures for denoising, okay? For noise prediction. Now, what is the role of U-Net in the DDPM pipeline or where exactly how exactly it is used? We will see it here. So, this is the image as we all know. This is the image for the forward diffusion where we start from X_0 , X_1 , X_2 to X_T , capital T we go where X_0 , this is my X_0 .

[04:25] This is my X_T . This is my $X_T - 1$, so on and so forth. X_2 , X_1 , so on and so forth, okay? Now, this is my reverse diffusion where this is my X_T , $X_T - 1$ and it goes on like, you know, these are all X_2 , X_1 , X_0 , okay? So, this is the reverse entire trajectory from X capital T to X_0 . This is known to all of us. So, the role of U-Net is at each and every reverse step. It predicts the noise, epsilon, which depends on X_T at T , okay?

[05:11] So, X_T goes inside the U-Net and it predicts the noise and in X_T and then it generates $X_T - 1$. It predicts the noise, that amount of noise is, you know, taken out to generate $X_T - 1$, okay? Now, we will have a quick look into the original architecture of U-Net. So, I have taken the picture from the original paper by Olaf Ronneberger. The original paper of U-Net Convolutional Neural Network biomedical image segmentation. So, this is a very known picture of unit architecture and we give an image as an input and we expect the unit to give the output segmentation map. Okay, so that is the usual usage of unit architecture. And if we see see the structure, we have the encoder, which is the path or the contracting path where you down

[06:13] sample and the size of the image becomes uh smaller uh spatial dimensions

become smaller and smaller keeping the important informations intact. Okay, so this is the encoder path. This is the decoder path or the expanding path where the image once again gets up sampled and reconstructed, you know, with the uh you know, with the all the inform important informations added to them via the skip connections. Okay? And this is the place which is the bottleneck and bottleneck is the region of unit architecture or it is uh the the the region where you get the most compressed information or all the important information in the most compressed form. Okay, so that's the, you know, bottleneck.

[07:08] And then these are the skip connections. You know, a portion whatever gets lost while the image is down sampled or the size becomes reduced, those a portion of that gets cropped and it gets concatenated from here to here through the skip connections and that is the way we keep the finer details, okay, in the image. So, this is basically the skip connections. So, these are the skip connections. Okay? So, this is the original unit architecture. But, DDPM uses a modified unit architecture because here in DDPM, our main goal is to not get the segmentation map of any image, but our goal is to find out the noise that is present at each and every time step so that that noise can be taken out of the image to generate the

[08:10] next image, okay? So, the DDPM used in uh sorry, the unit used in DDPM architecture, it has three things. One is the time step embeddings, residual blocks, and attention layers. So, we will see how the unit has been modified, what are the advantages of these modifications, and how the time step information is incorporated in the unit architecture's output. So, we will first see the time embeddings in DDPM unit. What exactly it is?

[08:43] So, we know time embedding means what exactly I mean? It means that when we are going coming from say x_t to x_{t-1} dot dot dot x_{t-2} then x_1 x_0 like this. At any instant of time, this is my time step t . This is my time step equal to t . This information is added with the input x_t to the unit. So, unit gets two information. One is the x_t , which is the noi- noisy image at the time step t , and the information about the time step.

[09:33] Why do we require this one? We require this one because the time step information indirectly indicates the noise level. If we remember when we started originally discussing about the forward diffusion process, we told that the clean image gets, you know, you know, noise is added to the clean image in a sequence, in a schedule, and the initial steps get lesser amount of noise compared to the I know, later stages. So, higher the T value, higher is the noise, okay? So, without the information about the time instant, the network will only see X_T , which is the noisy image, and it will not know from which time instant it has come. Whether it has come from, uh, you know, T equal to zero, T or T equal to five, or T equal

[10:32] to 1,000, okay? So, but that information is very important because it indi-

rectly indicates the noise level that can be present in that image. Okay? So, just for example, say T equal to 50. It is slightly noisy image. T equal to 500, moderately noisy image. T equal to 900, it may be a very heavily noised image, okay? So, at each training step, the DDPM provides x_t and T to the unit architecture. So, these goes as the input to the unit architecture. Where x_t is the noisy image, and T is the diffusion time step.

[11:09] Now, this time step, we don't give it as a scalar, okay? So, what we do is we convert the time step into a time embedding vector, okay? So, how it is done? So, instead of feeding the scalar T directly, it is that that information, time information, is converted into a high-dimensional vector using something called a sinusoidal embedding, which is a very similar concept in transformers, okay? So, we will see what is this sinusoidal embedding and how we get the embeddings from the time information, okay?

[11:48] So, for embedding in dimension D , time embedding T you know, $2 \lfloor D/2 \rfloor$ equal to $\sin(T/10^{\lfloor D/2 \rfloor})$ and $2 \lfloor D/2 \rfloor + 1$ equal to $\cos(T/10^{\lfloor D/2 \rfloor})$ exponential of $2 \lfloor D/2 \rfloor$, okay? Or 10 to the power of $4 \lfloor D/2 \rfloor$, okay? So, these are the two equations that are used for you know, getting the time embedding. So, let us have a look.

[12:32] What is this D ? The question that comes is D . Now, D can be 0, 1, 2, okay? Dot dot dot. So, typically implementations use embedding dimensions between 128 to 512. And based on that one, our D value will vary. It will always start from 0 till what it will go will depend on the dimension that is implemented, okay? Now, each D will give two terms. One is this $2 \lfloor D/2 \rfloor$ another one is $2 \lfloor D/2 \rfloor + 1$. To give a more you know, better clarity, let us have a numerical that D will solve for you all here. So, suppose Suppose we take T equal to 100.

[13:23] And I take the dimension D equal to 8. Now, for D equal to 0 10 to the power of $4 \lfloor D/2 \rfloor$ can I write it as $2^3 4^2$ into 0 by 8 that is equal to 10 to the power of $4 \lfloor 0/2 \rfloor$ which is equal to 1. Okay? So, the argument which is inside your sign or cos becomes you know 100 by 1 which is your 100. What was that one? It is T by 10 to the power of $4 \lfloor D/2 \rfloor$. Okay? So, now your $2 \lfloor D/2 \rfloor$ is your 1.

[14:27] Uh sorry, $2 \lfloor D/2 \rfloor$ is your 0. So, this entire term becomes 1 which is put here. Okay? So, this is like this. Okay. Now, time embedding for $2 \lfloor D/2 \rfloor$ is equal to time embedding $2 \lfloor D/2 \rfloor$ value is now 0. Okay? That is equal to $\sin(100)$ which is almost equal to minus 0.506. You can cross check it. If there is any small typo or something please bring it to my notice. Time embedding for $2 \lfloor D/2 \rfloor + 1$ sorry $2 \lfloor D/2 \rfloor + 1$ 1 minute. Yes. This is $2 \lfloor D/2 \rfloor + 1$ that is your time embedding one.

[15:43] That is equal to cos of 100 which is equal to 0.862. Okay? Similarly, for D

equal to 1, what is the value? 10 to the power of $4/2$ I by D becomes $3/4/2$ into 1 by 8 that is 10 to the power of $4/1$ by 4 equal to 0.25 . Okay? So, which is equal to 10 . So, here now the argument becomes 100 by 10 that is equal to 10 . Okay? Now, time embedding for I equal to 1 that is 2 into I equal to time embedding 2 into 2 . Sorry, 2 into 1 .

[16:57] That is time embedding two. What are we getting? That is $\sin 10$ which is equivalent to 0.544 . Similarly, time embedding for $2I + 1$ is equal to time embedding for three that is $\cos 10$ which is equivalent to $\sin$ of 0.839 . Okay. Similarly, if you do for I equal to two we get T_4 T_5 and for I equal to three we get T_6 and T_7 . Sorry, TE_6 and TE Everybody will be TE . Okay. TE_6 and TE This is seven. Okay. So, therefore, totally we get TE_0 TE_1 TE_2 TE_3 dot dot dot dot dot TE seven.

[18:21] Therefore, time embedding for 100 , which was my capital which was my time step value, can be written as a vector of all these values 0.506 , 0.862 , -0.544 -0.839 dot dot dot dot dot Okay. So, this becomes your eight-dimensional vector which which is fed to your unit architecture. Okay. So, that is the way the time embeddings are calculated, okay? I hope that makes it clear, okay? Now, once the time embeddings are done, we will now see what exactly we know in the unit. We have the encoder side, the bottleneck, and the decoder side. So the time embeddings are ready with us. We will keep it aside for time being, and we will see how the encoder, bottleneck, and the decoder parts of the unit architecture have been modified

[19:41] in the DDPM structure for predicting the noise, okay? So we will see the encoder side of the unit. We all know encoder is known for extracting increasingly abstract features from the image, okay? So it extracts all the important information that is present in the um image, you know, discarding all the unimportant things, okay? So the noisy image x_t goes to uh So this is the input to the, you know, unit, and then in the encoder side, and then we have the convolution layers.

[20:17] Then it This from the output of this convolution layer goes to residual blocks, another residual block, then there is a down sampling, then time embeddings go in each of the resi- So whenever there is a residual block, the time embedding goes inside the residual block, and we will see in my uh one or two slides after this one we will see exactly where in the residual block this time embedding is going and sitting, okay? Or how the calculations inside the residual block is happening, which embeds the time information, okay?

[20:52] So we have res- from conv layer we have a feature map, which which comes to two residual blocks, and then there is down sampling. Again, there is residual block plus attention. So there is an attention mechanism also that is present in this unit architecture, which is used by DDPM for predicting the noise. And wherever there is residual block, there is time embedding. Another another block of residual uh you know block plus attention, and this is the full encoder side of the unit. Okay?

[21:29] Now, after the residual block, there is another I'm sorry, after the residual block, there is another down sampling, and this is the full encoder side of the unit. Okay? Now, this entire thing is your encoder. Now, what is the function of the residual block? The residual block learns features useful for noise prediction. The receipt inside the residual block, we will see the time information gets embedded, and this is helpful for prediction of the amount of noise that is present in each and every step, okay, of the noisy image.

[22:13] Then we will see what is there after the encoder. We have a bottleneck. So, in the bottleneck, we have a residual block, attention block, again another residual block. Okay? So, that is my bottleneck and time embeddings as usual keeps, you know, gets incorporated in the residual blocks. Then we will see the decoder block. What is there? So, we all know in the decoder block that is uh up sampling part or the you know expanding path we call in the unit architecture. So, it starts with up sampling, then concatenation of the skip connection happens here. The skip connections come from the encoder region and gets concatenated in the uh portion of the image in the decoder part, okay? So, then there is another

[22:58] one layer of res block plus attention, another layer of res block plus attention, then another layer of up sampling, then concatenation skip of skip connections here, then another res block, another res block, and here once again that time step embeddings happen whenever there is a res block, okay? Residual block. So, it goes like x_t is your noisy image. It goes to the encoder through the and then through the bottleneck, then decoder, and what we get is the predicted. This is your predicted noise.

[23:31] So, this is your predicted noise. Okay? Now, we will have a look into the entire simplified DDPM unit architecture, how it looks. So, if you see, here is your noisy image x_t . It goes through the convolution layer. This entire block is your encoder block, this entire thing. Then you go to the bottleneck block. This is full your bottleneck. Uh I can put it here. After the bottleneck, it goes to the decoder block. So, the entire decoder block, whatever I have shown in my previous slide, that is here, and then it goes to the concatenation of the initial, you know, skip connection from here. Then we have a final convolution layer, and then we get the predicted noise.

[24:25] Okay? So, this is the simplified DDPM unit architecture. That is the, you know, these are the skip connections, that is a concatenation of the skip connections. So, these are coming from the encoder block and getting, you know, um uh what what it it gets concatenated with the feature maps of the decoder block, okay? So, these are the skip connections. So, the skip connections up in the skip connection, what happens? A portion, small portion of the encoder block which is you know uh having the finer details goes from there crop and copy and then it goes and concatenates concatenated with the feature map in this decoder block same layer same level and then we keep on up sampling okay so this is the unit

[25:18] architecture that is used in the DDPM okay now what happens in the residual block we have to now see what happens inside the residual block okay we will have a look so we know there is an input H this input H is basically your feature map which have the dimensions $B \times C \times H \times W$ where B is the batch size C is the number of channels $H \times W$ is the spatial size okay now this feature map goes to group normalization and we get $\hat{H}$ which is your H undergoing group normalization okay and after that what happens it goes through an activation called SiLU so this $\hat{H}$ will now go through SiLU activation function and it will give you A now what is the mathematical form of SiLU activation function it is X into

[26:27] $\sigma(X)$ okay so that is the mathematical expression for SiLU activation function then this A goes through conv one so it goes through convolution layer one okay so there is so this A goes to conv one and we get something H_1 okay after H_1 there is a big step which is processing the time step embedding. We have Previously, we have done the time step embeddings, kept it aside, and now that those embeddings we will use here inside the residual block. Okay?

[27:10] So, what we do here? Here, the time step is converted into sinusoidal embedding. So, whatever we have done there, actually that is done here first. And then there is a step called time projection where this operation happens. So, BT equal to WT into the time time embedding for T . Where WT is a learned linear layer. So, this is the you know, your time embeddings are getting projected. So, projection time projection happens in this operation.

[27:46] Okay? And then once we get VT , we add this one to H_1 , which one is which is your the output of after the convolution layer. So, we add VT to that one and we get H_2 . Okay? And then this is continued. I will In the next slide. So, we will go here. So, this is the process time step embedding, whatever we have done till now. And then the output again goes to the group normalization and we get H_3 equal to group normalization of H_2 . H_2 is equal to H_1 plus the BT , okay? Which is coming from the time projection.

[28:35] Then again, Silu activation and we get H_4 , which is Silu of H_3 . Then we have another convolution layer which gives you H_5 convolution 2. H_4 goes to that convolution layer. And then we have something the residual connection. Okay. So, what exactly happens here? So, the H output will be capital H plus H_5 . So, H was here initial feature map that came inside. And the output will be basically H out H out equal to H plus f of H . What is f of H ? f of H is equal to H_5 , which is basically is your learned transformation.

[29:23] So, all these steps that we have done that have given me H_5 and then in the residual connection H out is an addition of your initial feature map plus f of H . Okay. So, this is what happens in the inside the residual block. That means the time step embeddings or the time step information gets encoded in the architecture. Okay.

Now, in some of the blocks I have shown that attention is used together with the either the rest block or separately.

[30:07] Now, what is this self-attention in DDPM unit? Why do we require it and how it is done? So, we'll have a quick look there. So, if we take only convolution layers, it sees only the local neighborhood. Whereas a pixel may need information from distant regions of the image. For example, as I was telling, to denoise the eye of a face, the network may need information from the other eye located far away in the image. So, there are two eyes.

[30:41] But there is a distance between one eye and the other. If we look only at the pixels which are very near to this eye, say left eye, and the information is required denoising information is required from the right eye, convolution layers will may not be able to give that one. For that one, self-attention has been implemented, okay? So, it gives attention to all the relevant parts and picks the information from there.

[31:12] So, how self-attention works? So, suppose we have a feature map X , which is of spatial dimensions H cross W . C is the number of channels, okay? And then from there we compute the query vector, the key vector, and the value vector, which are the build which are the main building blocks of self-attention, okay? And so, we have the query vector Q equal to XW_Q , K equal to XW_K , V equal to XW_V , W_Q , W_K , W_V , these are the learnable weight matrices, okay? For W_Q is for query, W_K is the learnable weight matrix associated with the key, and W_V is the learnable weight matrix associated with V , okay? With the value, okay?

[32:09] So, once we get the query, key, and value, we compute the attention score, attention weight, which is given by softmax of $Q \cdot K^T$ divided by square root of D , where D is the dimension here, okay? Uh it's basically a scalar division by a scalar, and then we get the output Y equal to $A \cdot V$. V is the value vector, okay? So, this is the way self-attention is used in unit that is used in DDPM for denoising uh images, okay? Or predicting noise in a certain image, okay? So, with this one I come to the end of my session today.

[32:56] Let's quickly go through whatever we have learned. The session was completely on the unit architecture that is used for um uh denoising images in denoising diffusion probabilistic models. And we have seen how the unit archi- architecture, the original one the unit original unit is used for uh was used for uh getting the segmentation map from for an image, but here in DDPM the unit is modified to include time embeddings, to include residual blocks, and attention. So, these are the three things self-attention. These are the three things that has been added in the unit architecture or modifications have been done in that direction, so that it can be used for noise level prediction, okay? And we have seen

[33:50] how the time embeddings are calculated, why it is required, why it is required? Because the time information of uh the noisy image is very important because

it indirectly it uh you know, brings the information how much noise can be there in that image. So, a low uh time instant and a very large time instant images, they will vary in the amount of noise that is present. Low time instant will have less noise, high one will have more noise. Okay, so that information is very important, otherwise the architecture of the neural network will not understand what from which uh place the um or which time stamp the information has come, okay? So, that is important. So, that has been taken care by the time

[34:39] embeddings. And we have seen through our numerical example how the time embeddings are calculated using this sinusoidal embedding equations, okay? And then we have seen how the residual blocks are added in the encoder part. Residual as well as attention blocks are added in the encoder part, in the bottleneck, and in the decoder part of unit architecture. And then we have seen what is there inside the residual blocks, why it is important. Residual blocks take the time information. Then what exactly happens inside the residual blocks so that the time information gets embedded in the architecture, okay? So, that we have seen. And finally we have seen how the self-attention is used in the unit that

[35:28] is used in DDPM. And we have seen if we have a feature vector, how the query, key, and the value vectors are calculated from there. And then finally the attention weight is calculated from there, okay? So, with this one I come to the conclusion of today's session. I hope you all have understood uh how the unit have been modified to be used for noise prediction in DDPMs. And in my next session I will uh discuss about classifier guided diffusion.

[36:03] And till that time, keep learning. Thank you all. »